

Proyecto: PoliticalSAT

Estabilidad de coaliciones políticas usando SAT

1. ¿Qué hace el programa?

Tenemos un parlamento con **n partidos**. Cada partido tiene una cantidad de **escaños** y una **postura** sobre **p políticas**. La postura puede ser:

- **+1** → el partido apoya esa política
- **-1** → el partido rechaza esa política
- **0** → el partido es neutral (no le importa)

El programa decide cada política como **+1** o **-1** y busca que los partidos que acepten **todas** esas decisiones junten al menos el **percentage%** de los escaños. Si no existe ninguna forma de lograrlo, reporta que es imposible.

2. ¿Cuándo un partido apoya la decisión?

Un partido apoya la decisión completa si, para **cada una** de las políticas, su postura es **neutral (0)** o **coincide exactamente** con la decisión tomada. Si en cualquier política el partido tiene opinión y esta no coincide, ese partido **no apoya** el gobierno.

Ejemplo rápido:

```
Decisión tomada: [ +1, -1, +1 ]
Partido A postura: [ +1, 0, +1 ] → APOYA (coincide o es neutral)
Partido B postura: [ 0, +1, +1 ] → NO APOYA (política 2: quería +1, decidimos -1)
```

3. Estrategia del algoritmo

Se prueban **todas las combinaciones posibles** de decisiones. Con **p** políticas hay 2^p combinaciones (cada política puede ser +1 o -1). Para cada combinación se cuenta cuántos escaños la apoyan. En cuanto se encuentra una que alcanza el mínimo, se

retorna. Si ninguna funciona, se retornan todos ceros (señal de imposible).

```
Para cada combinación de decisiones posible:
    escaños_a_favor = 0
    Para cada partido:
        Si el partido apoya toda la decisión:
            escaños_a_favor += escaños del partido
    Si escaños_a_favor >= mínimo necesario:
        devolver esta combinación ← ÉXITO
devolver todos ceros ← IMPOSIBLE
```

4. El código — solver.py

__init__: guardar los datos

```
def __init__(self, seats, policies):
    self.seats = seats      # escaños de cada partido
    self.policies = policies # posturas de cada partido sobre cada política
```

solve_stability: buscar la decisión ganadora

```
def solve_stability(self, percentage):
    n = len(self.seats)      # cantidad de partidos
    p = len(self.policies[0]) # cantidad de políticas
    total = sum(self.seats)   # total de escaños
    minimo = total * percentage # mínimo para gobernar

    # Probamos todas las combinaciones: (-1,-1,...), (-1,-1,...,1), ...
    for decision in product([-1, 1], repeat=p):
        escanos_favor = 0

        for i in range(n):      # revisamos cada partido
            partido_apoya = True
            for j in range(p):   # revisamos cada política
                postura = self.policies[i][j]
                # Si tiene opinión y no coincide con la decisión → no apoya
                if postura != 0 and postura != decision[j]:
                    partido_apoya = False
                    break
            if partido_apoya:
                escanos_favor += self.seats[i]

        # ¿Alcanzamos el mínimo?
        if escanos_favor >= minimo - 1e-12:
            return list(decision) # éxito

    return [0] * p # imposible
```

generate_dnf: expresar las condiciones en lógica

La DNF (Forma Normal Disyuntiva) describe cuándo cada partido apoya la decisión. Cada partido genera una cláusula AND con sus restricciones, y todas se unen con OR.

```
def generate_dnf(self):
    clausulas = []
    for i in range(len(self.seats)):
        literales = []
        for j in range(len(self.policies[0])):
            if self.policies[i][j] == 1:
                literales.append(f'x{j+1}') # quiere política j = +1
            elif self.policies[i][j] == -1:
                literales.append(f'!x{j+1}') # quiere política j = -1
            # si es 0, no agrega nada (neutral)
        if len(literales) == 0:
            clausulas.append('T') # acepta todo
        else:
            clausulas.append('(' + ' ^ '.join(literales) + ')')
    return ' | '.join(clausulas)
```

5. Ejemplo paso a paso (primer caso de in_01.txt)

5 partidos, 5 políticas, necesitamos $\geq 50\%$ de 59 escaños = 29.5

Escaños: [19, 8, 16, 1, 15]

Posturas:

```
Partido 0: [ 0, -1, -1, 0, 0]
Partido 1: [-1, 0, -1, 1, 0]
Partido 2: [-1, -1, -1, 0, -1]
Partido 3: [ 1, -1, 0, 1, 1]
Partido 4: [-1, 1, 1, 0, -1]
```

Primera combinación probada: (-1, -1, -1, -1, -1)

```
Partido 0: neutro en 1,4,5 | política2=-1==dec ✓ | política3=-1==dec ✓ → APOYA (19)
Partido 1: política4=+1 ≠ dec=-1 → NO APOYA
Partido 2: todas coinciden o neutral → APOYA (16)
Partido 3: política1=+1 ≠ dec=-1 → NO APOYA
Partido 4: política2=+1 ≠ dec=-1 → NO APOYA
```

escaños_favor = 19 + 16 = 35 $\geq$ 29.5 → ÉXITO → retorna [-1, -1, -1, -1, -1]

6. Resultados de las pruebas

Se corrieron los 5 archivos de prueba del proyecto. Todos pasaron correctamente:

- **in_01.txt** → ✓ correcto (< 1 ms)

- **in_02.txt** → ✓ correcto (~213 ms)
- **in_03.txt** → ✓ correcto (~82 ms)
- **in_04.txt** → ✓ correcto (~3.2 s)
- **in_05.txt** → ✓ correcto (~125 ms)

Comando para probar en Windows:

```
python main.py "-read_case=cases\in_01.txt" "-print_cmp=stdout"
```